Seeing Through the Canopy

Recovering apple-tree leader structure from summer RGB-D images

MMA3001 - Numerical Methods and Machine Learning S2- 2026
github.com/shreejikumawat/MMA3001

Student Name: SHREEJI KUMAWAT **Student ID: 37573578**

---

0. ABSTRACT

This project looks at apple trees in an orchard and tries to find the trunk and the main branches even when leaves are covering them. A small neural network called a U-Net was trained to do this from a colour and depth photo of each tree. It was then tested on rows of trees that it never seen during training. The network did a lot better then a simple rule based method that was used for comparison. It also measured the angle between the main branches quite closely and it runs really fast on a normal laptop. While checking the data we found that the hand drawn answer masks are shifted sideways from the real trees in most of the images. This makes every strict score look lower than it should be and so a second score that allows for this shift was also reported. Smaller images worked better then bigger ones which was a bit suprising. The height where the trunk splits was not measured as well as was hoped and this is still a weak point of the method.

---

1. ENGINEERING PROBLEM

A robot that prunes apple trees has to know where the trunk and branches are before it moves its arm into the tree. In a summer lot of this is hidden behind leaves which is our main focus point here. In this dataset about 37% of the labelled branch centreline is covered by leaves (Section 2).

CONTEXT: The orchard uses a 2D open-V trellis. This means each tree has a short trunk that splits into two main branches called leaders and these are trained into a flat V shape. Because the tree is flat a robot arm can reach the whole tree from the gap between the rows. To plan its movement the robot needs two numbers for each tree. These are the height where the trunk splits (the fork height) and how wide the V opens (the leader angle). It also needs a rough 3-D line of the branches so it can avoid hitting them.

PROBLEM AND WHY IT MATTERS: The problem is to take one summer colour and depth photo of a tree and find the trunk and leaders including the parts that are hidden. From this the fork height, the leader angle and a 3-D skeleton is reported. This matters because pruning and thinning by hand is one of the biggest labour costs for apple growers and it is getting harder to source workers. Most methods only work in winter when there is no leaves. If the structure can be found in summer too then the same robot can help with summer pruning and thinning as well.

INTENDED USE: The tool is meant to run once per tree on the robots computer before it plans a path. It has to be accurate enough that the arm does not hit a leader. It also has to be fast enough to keep up as the robot moves down the row and the target for this was well under 1 second per tree on a normal CPU.

SCOPE AND LIMITS: Only summer images are used as inputs. The winter images were only used for checking the data. Also only the main structure is found which is the trunk and the two leaders. This is because that is all the answer masks have labelled so small side branches are left out. The 3-D output is only relative and not in real units because the depth images are 8-bit and their scale is unknown. This method also assumes the tree is a 2D open-V shape and the measurements do not always necessarily have to make sense for other tree shapes.

---

2. INPUTS | OUTPUTS | DATA

The inputs and outputs of each function in the code are explained in the docstrings. These are written in Google style and they were also turned into HTML documentation using pdoc.
DATASET: The dataset used is the TreeID dataset and it was given by the teaching team. Each tree has five files. These are a summer colour image, a summer depth image, a 256 × 256 summer answer mask (called the ground truth or GT), a winter colour image and a winter depth image. The images were cropped to the middle 480 × 480 square and then resized, the same way the original study did it. None of this data is uploaded to GitHub repository and instead the file data/README.md explains how to get it.

Table 1: Inputs and outputs

Item	Format	Units and domain	Meaning and use	Invalid or missing input
Summer RGB (in)	PNG, 640 × 480 × 3, uint8	0 to 255 per channel	Colour image of one tree from the inter-row	Wrong shape or unreadable file raises ValueError
Summer depth (in)	PNG, 640 × 480, uint8 (3 identical channels)	Relative; larger = farther; 0 = no return	Separates the tree from the next row	Zeros kept as missing and passed as a separate validity channel
Branch probability (out)	float32, H × W	0 to 1, over the 480 × 480 centre crop	Per-pixel belief that trunk or leader is present, visible or not	n/a
Branch mask (out)	bool, H × W	Probability above a threshold chosen on validation rows	Region the arm must avoid	n/a
Fork row (out)	int	Pixel row, 0 = top	Height of the trunk split	None if no qualifying junction
Leader angle (out)	float	Degrees, 0 to 180	Opening of the V	None if a leader has < 10 skeleton pixels
3-D skeleton (out)	(K, 3) float	X, Y relative; Z in raw 8-bit depth units	Centreline for path planning	Pixels with no valid depth in a 7 × 7 window dropped

Before any modelling the full data was checked. The findings, and what each changed, were:

Table 2: Data check findings

Check	Finding	Consequence
Depth direction	Canopy reads larger than ground at the bottom of the frame in 99.6% of trees, so larger = farther	Depth rank-normalised with near = 1; the initial "near = bright" assumption was wrong
Missing depth	16% of summer pixels have no return	Validity channel added
Registration	GT displaced horizontally from the visible trunk; best-fit offsets have median magnitude 10 px, both signs	Strict scores are capped; a shift-compensated score is reported beside them

---

3. COMPUTATIONAL APPROACH

Two methods were built and compared. The first one is a simple rule based detector with four settings that describe what a branch looks like. The second one is a small U-Net that learns what a branch looks like by itself and it can also guess where the hidden leaders go. Both methods then feed into the same step that measures the tree.

3.1 PREPROCESSING (PREPROCESS.PY)

The images are resized to 64, 128 or 256 pixels wide. The colour and depth images are shrunk by averaging the pixels together. This stops the small details from turning into noise when the image gets smaller (unit week 3, interpolation). The answer masks are resized by just copying the nearest pixel so no new label values gets made up. Pixels with no depth are kept marked as missing. The rest of the depth values are ranked inside each image which gets rid of the unknown scaling in the 8-bit depth. A greenness value called excess green (ExG = 2g − r − b) is also worked out. This seperates leaves from bark and it does not depend on how bright the photo is.

3.2 BASELINE (BASELINE.PY)

First any pixel that is too green (a ExG above 0.05) is removed because it is probably a leaf. Then each pixel gets a score that mixes two things. The first is how much it looks like a thin dark line (from a ridge filter) and the second is how close it is to the camera from the depth. If the score is above a threshhold the pixel is marked as branch. Small blobs are removed at the end. The four parmeters were tuned by trying 75 combinations on half of the training trees and picking the one with the best centreline F1. This method is fast and easy to understand and it does not need any training data. Its main limit is that it can only find bark that you can actually see.

3.3 MAIN METHOD (MODEL.PY, TRAIN.PY)

Network: The U-Net (Ronneberger et al., 2015) shrinks the image 3 times and then grows it back. It has about 0.48 million parameters and needs 0.29 GFLOP for each 64 px image. It was kept small because there is only a few hundred labelled trees and it has to run on a robot's CPU.

Loss: The training loss is binary cross-entropy plus a Dice loss. The cross-entropy gives extra weight to branch pixels. This is needed because the branches is only 7.5% of all pixels. Without it the network just learns to predict background everywhere.

Training settings: The optimiser is AdamW with a learning rate of 0.002 and a batch size of 8. Training runs for at most 40 epochs. It stops early if the validation score has not improved for 8 epochs and then the best weights are kept.

Flips and threshold: The training images are flipped left to right only. Flipping them upside down would show trees growing downwards which never happens. The cut-off for calling a pixel branch is picked on the validation rows. All random seeds are fixed and the main runs were done 3 times with different seeds.

3.4 MEASURING THE TREE (SKELETON.PY)

The predicted mask is thinned down to a one pixel wide line called a skeleton. The fork is the lowest point where the skeleton splits into two pieces that are both at least 20 pixels long. Above the fork the skeleton is split into a left and a right leader. Each leader are fitted with a straight line using least squares (unit week 5) and the angle between the two lines is the leader angle. The line is fitted as column against row because the leaders are close to vertical and this keeps the fit stable. For the 3-D skeleton each pixel is moved into 3-D using the pinhole camera model with the depth taken from the pixels around it. The depth scale is unknown so the 3-D result stays in relative units.

Key assumptions:

1. The middle 480 × 480 crop of the photo lines up with the area of the answer mask.
2. A bigger depth value means further away and 0 means missing. This was checked in the data audit.
3. Trees in the same row are similar to each other so the data was split by whole rows.
4. An ExG above 0.05 means leaves. This is only used for the occlusion numbers and the baseline and the U-Net does not use it.

4. VALIDATION AND VERIFICATION DESIGN

What is validated: The predicted branch masks are checked against the hand drawn answer masks on test trees that were kept aside. The fork height and leader angle from the predicted mask are compared with the same values worked out from the answer mask. How well the method works across the whole orchard is checked by splitting the data by rows. The code itself is checked with tests that has known answers.

Splitting by orchard row (splits.py): Trees in the same row share the same lighting, background, camera position and time of day. If the images were split randomly the model could learn these things and the scores would look better than they really is (unit week 5.6). So whole rows were put into one group only. This split was fixed in the code before any training was done.

Table 3: How the data was split

Partition	Rows	Trees	Used for
Training	the other 14 rows	398	Fitting weights; tuning baseline parameters
Validation	3, 13	67	Early stopping, threshold, and model selection
Test	5, 10, 15, 20	97	One final evaluation of each model

The validation rows did affect the model choice. They were used to pick when to stop training, the cut-off threshold and the final model (Section 6.1). The test rows was never used for any of this and the test labels were never changed.

Metrics: All scores are worked out at the answer mask size of 256 × 256. This way models trained at different image sizes are compared fairly.

Centreline F1 at 4 px (main score): Both the predicted mask and the answer mask are thinned to one pixel lines. A true line pixel counts as found if there is a predicted line pixel within 4 pixels of it. The same check is done the other way round for precision. The branches are about 11 pixels wide so 4 pixels means the predicted line still stays inside the real branch.

Shape F1: This is the same score but the answer mask is first allowed to slide sideways by up to 24 pixels to wich ever position fits best. It shows if the predicted shape is right even when the label is shifted. This score is always a bit too kind so it is always shown next to the strict score and never by itself.

Other scores: Occluded branch recall shows how much of the branch hidden behind leaves was found. The leader angle error is in degrees and the fork height error is in pixels.

Checking the code: There are 53 tests written with pytest and they all pass (doc/test_report.txt). Each test checks a part of the code against a case where the right answer is already known.

Table 4: Code tests

Component	Manufactured or analytical check
IoU, Dice, tolerant F1	Scores worked out by hand match the code. A line moved by fewer pixels than the tolerance still counts as a match
Leader angle	Fake V shapes with known angles give back the right angle to within 1.5°
FLOP counter	Matches a FLOP count done by hand for one layer and goes up 4 times when the image size doubles
Training loop	Learns a very simple fake task, and two runs with the same seed give exactly the same result

Pass criteria: These were set from what the robot needs before the final runs were done.

1. The U-Net has to beat the baseline on centreline F1 with a paired p-value below 0.05.
2. The median leader angle error has to be under 5° and the median fork height error under 10 pixels which is about 4% of the image height.
3. One tree has to be processed in under 1 second on one CPU thread.
4. The centreline F1 should not change by more than 0.03 between diffrent random seeds.

What this cannot show: These checks cannot show the true accuracy because the answer masks are shifted from the real trees. They also cannot check the 3-D output in real units since there is no depth scale. All the data comes from one orchard and one camera so it is not known if the method works on other orchards or cameras. It was also never tested on a moving robot.

---

5. RESULTS AND COMPARISON

The U-Net numbers in the table are the average of 3 runs with different random seeds and the ± shows how much they changed between runs. The baseline always gives the same result so it only has one number. The numbers in brackets are 95% confidence ranges worked out over the test trees for the first seed. The inference time is how long the network takes for one image on one laptop thread (scripts/profile_pipeline.py).

Table 5: Main results on the test rows

Method (test rows, 97 trees)	Centreline F1, strict	Shape F1	Occluded recall	IoU	Mean angle error	Inference
Classical baseline	0.120 [0.114, 0.125]	0.159	0.54	0.055	31.7°	65 ms
U-Net RGB-D, 128 px	0.257 ± 0.009	0.431 ± 0.024	0.81	0.225	7.8°	19 ms
U-Net RGB-D, 64 px (selected)	0.295 ± 0.007 [0.261, 0.323]	0.462 ± 0.006	0.87	0.260	6.3° (median 4.7°)	6 ms

Learning beats hand made rules, mostly for hidden branches: The baseline marks every dark line it see as a branch. This includes grass, shadows and the trellis wire so its precision is only 0.07. It also cannot see through leaves at all. The U-Net found 87% of the hidden branch line while the baseline only found 54%.

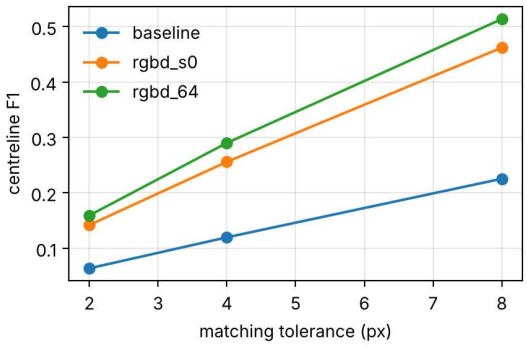


Figure 1. Score a perfect prediction receives when its label is shifted horizontally (97 test masks).

Reason behind strict scores being low: Figure 1 shows how quick even a perfect detector loses score when the label is shifted sideways. When the selected model was scored with the shape score the median shift came out at 7 pixels. On 68% of the test trees the shift was more then 4 pixels. This means the strict score is mostly measuring how well the labels line up and not how good the model is. So the shape score is a better way to tell if the predicted structure is actualy right. Figure 2 in Appendix A shows some trees where the model follows the real trunk but the hand drawn label do not (R05N01 and R15N08).

Table 6: Pass criteria results

Acceptance criterion	Target	Result (selected model)	Met?
Beats baseline	$p < 0.05$	+0.170, $p = 2 \times 10^{-14}$	Yes
Leader angle	median < 5°	4.7°, 4.7°, 4.9° (3 seeds)	Yes, narrowly
Fork row	median < 10 px	17, 19, 11 px (3 seeds)	No
Inference	< 1 s per tree	about 16 ms end to end (6 ms network)	Yes
Repeatability	seed SD < 0.03	0.007	Yes

6. ALTERNATIVES, OPTIMISATION AND PERFORMANCE

6.1 Optimisation path

The first working model used 128 px images (v1, 128 px). It was then compared with the exact same model using 64 px images. The 64 px version was better on every accuracy score and it also took half the time to run so it was picked as the final model.

The 64 px result was first spotted when looking at the test scores. Picking a model from test scores is not allowed because it uses the test set for tuning. So the choice was checked again using only the validation rows (scripts/validation_selection.py). The 64 px colour and depth model came first there too with a strict F1 of 0.290 against 0.250 for the next best. This shows the choice do not depend on the test set.

Table 7: First model vs selected model

Version	Change and reason	Strict F1	Shape F1	Decision
v1, 128 px	Weighted loss so the small branch areas count more	0.257 ± 0.009	0.431 ± 0.024	Starting point
v1, 64 px	Smaller input images (see 6.2)	0.295 ± 0.007	0.462 ± 0.006	Selected

6.2 Input resolution

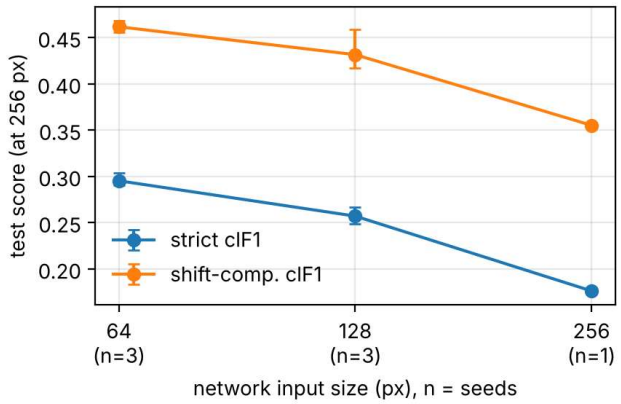


Figure 3. Strict and shape-compensated centreline F1 at three input sizes (mean over seeds, bars = min to max). All are scored at 256 px.

The accuracy gets worse as the images get bigger. The strict F1 is 0.295 at 64 px, 0.257 at 128 px and only 0.176 at 256 px. There is two likely reasons for this. The first is that the network shrinks the image three times. With a small 64 px image each part of the network sees a big chunk of the tree so it learns the whole V shape instead of small bark details. The second is that a 10 pixel label shift at 256 px is only 2.5 pixels at 64 px. So the smaller images gives the network less confusing answers to learn from.

6.3 Sensitivity

The scores were also worked out with a 2, 4 and 8 pixel tolerance. The selected model scored 0.16, 0.29 and 0.51 and the baseline scored 0.06, 0.12 and 0.23. The U-Net wins at every tolerance so the result does not depend on which tolerance is choosen.

6.4 Runtime, FLOPs and the roofline

One tree was timed on one thread using cProfile and timers (results/profile.json). Reading the two PNG files took 9.3 ms (57%) and cropping and resizing took 0.6 ms (4%). The U-Net itself at 64 px took 6.0 ms (36%) and finding the fork and angle took 0.6 ms (3%). This was all run on a laptop with Windows 11 and an Intel CPU with 10 threads.

The network needs 0.29 GFLOP for a 64 px image, 1.17 GFLOP at 128 px and 4.68 GFLOP at 256 px. These were counted by the estimate_flops function and checked against a count done by hand. The arithmetic intensity of the network is between 43 and 58 FLOP per byte. The laptop can do 111 GFLOP/s and move 23.3 GB/s of memory so its balance point is about 4.8 FLOP per byte. These were measured with a matrix multiply and an array copy. Since the network is well above this point it is limited by calculation speed and not by memory speed (compute-bound).

The baseline takes 65 ms for each tree and most of this is spent in its ridge filter. So it is about 11 times slower then the U-Net and it is also less than half as accurate.

---

7. LIMITATIONS, FAILURE MODES AND LESSONS LEARNED

The biggest limitation in my opinion is the labels and not the method. Until the answer masks are lined up with the images they are scored against none of the numbers in this report show the true accuracy.

Label shift: The answer masks are shifted sideways by a different amount on each tree (about 10 pixels on average at 256 px). This means the strict scores are held down and the shape score is a bit too kind. As an extra check the real trunk was marked by eye at one row of the image (row 230 out of 256) on the 76 test trees where the trunk could be seen (results/trunk_check.json). The answer label was a median of 17.5 pixels away from the real trunk and the selected model was 14 pixels away. This check only looks at one row so it doesnt cover the whole tree.

Fork height misses its target: The fork height error was 11 to 19 pixels against the target of 10 pixels. The predicted fork is where the predicted leaders split apart and this depend on how wide the mask is. The predicted masks are still around twice as wide as the real branches so the fork ends up in the wrong place.

Relative 3-D only, one orchard and one camera: The camera settings used to make the 3-D skeleton was assumed. There is nothing to show the method works on other apple types, trellis shapes, lighting or cameras. The 256 px result also only comes from one run.

Unsupported inputs and failure modes: If a tree is not an open-V shape the tool still gives a fork and angle for the two biggest branches. It only returns None when no split can be found. If more than 30% of the depth image is missing the depth becomes useless and the model has to rely on colour only. A trellis post near the trunk gets marked as branch by the baseline and sometimes the U-Net joins it onto the trunk too. Images that are not 640 × 480 or colour and depth images of different sizes are stopped with a ValueError before anything runs and this is tested. Any different camera or crop is outside of what was checked.

Lessons learned:

1. The data should be checked before any modelling. The two most important findings were the depth direction and the label shift. Both came from checking the data and they changed how every result was read.
2. With labels that are a bit wrong bigger images made things worse. The smaller images gave a model that was more accurate and faster and more consistant between runs.

3. It is important to measure what the robot actually needs. Pixel overlap scores ranked the methods in a similar way but they said nothing about the angle or the fork which is what a robot uses to plan.
4.

Improvements: The answer masks could be lined up with the images by marking a few matching points on each tree by hand. The network could also be trained to predict the centre line directly instead of the full mask. This would stop the mask width from pushing the fork in the wrong spot. The winter images could also be used to make new labels since the branches are easy to see in winter and the labels would line up by default.

---

8. AI USE AND REFLECTION

Tools: Claude (Anthropic), used between 7 and 14 October 2026 and again on 23 October 2026.

Table 8: AI use and contribution

Task	AI contribution	My contribution
Checking code and fixing errors	High	Ran the code, shared the errors and used the suggestions to fix them.
Brainstorming and improving methods	Mid	Chose which ideas to use based on the problem and the results.
Checking results and assumptions	High (first drafts)	Checked the results against the data and the purpose of the project. Used AI in determining whether something looked right and behaved correctly.
Report writing	Low	Decided what to include in the final report and how to present the work.

Overall I estimate that AI wrote 80% of the code. It was also used to check the code, find errors and suggest fixes. I used AI because I wanted to spend more time understanding the methods, checking the results and improving the project rather than spending most of the time writing code from scratch.

Some of the errors and unhelpful output, and how they were caught:

- The first filename parser silently skipped the 42 renamed row-4 winter files. This was caught because the audit printed an "unparsed" count, and it was fixed and covered by a test.
- Its first registration reference, based on ridge texture, gave ambiguous peaks, and the colour reference was pulled onto trellis posts. Automatic label correction was therefore rejected.
- AI identified the 64 px model while looking at test results. The choice was checked again using only the validation rows before the model was selected.
- AI also suggested a journal name for the source paper that could not be verified. This was replaced with the arXiv reference.

How AI output was checked:

The code was checked by running the tests and examining the results of the experiments. The final results were compared across the baseline and the different U-Net input sizes. The validation rows were used to check the choice of the final model rather than relying on the test results alone.

Decisions that stayed mine:

The main decisions were the choice of dataset, the problem to solve, the decision not to automatically correct the answer masks, the acceptance criteria and the final choice of the 64 px RGB-D model. The choice of metrics and the way the results were interpreted were also based on what the robot needs. AI helped suggest and check different approaches, but the final report had to reflect what the experiments actually showed.

Effect on my understanding and workflow:

Using AI helped me understand why checking the data before training is important. The depth direction and the shift in the answer masks both changed how the results had to be interpreted. I also understood why the data should be split by whole orchard rows, since trees in the same row share similar conditions and a random split could make the results look better than they really are. Another important lesson was that a high pixel overlap score does not necessarily mean that the trunk and leaders have been found correctly. Next time I would spend more time checking the labels and the data before training the model, because these issues can affect the results more than changing the model itself.

I. REFERENCES

Chen, Z., Ting, D., Newbury, R., Chen, C. (2020). Semantic segmentation for partially occluded apple trees based on deep learning. arXiv:2010.06879. Source of the TreeID dataset and of the occluded-branch recall metric.
Ronneberger, O., Fischer, P., Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. MICCAI. arXiv:1505.04597.
Sato, Y. et al. (1998). Three-dimensional multi-scale line filter for segmentation and visualization of curvilinear structures in medical images. Medical Image Analysis 2(2), 143 to 168.
Woebbecke, D. M., Meyer, G. E., Von Bargen, K., Mortensen, D. A. (1995). Color indices for weed identification under various soil, residue, and lighting conditions. Transactions of the ASAE 38(1), 259 to 269.
Williams, S., Waterman, A., Patterson, D. (2009). Roofline: an insightful visual performance model for multicore architectures. Communications of the ACM 52(4), 65 to 76.
MMA3001 unit notebooks
Software: NumPy, SciPy, scikit-image, OpenCV, PyTorch, pandas, Matplotlib, pytest, pdoc.

II. APPENDIX A


R05N01 GT
baseline clF1 0.11 / comp 0.12
rgbd_s0 clF1 0.23 / comp 0.43
v2r_rgbd_s0 clF1 0.11 / comp 0.35
rgbd_64 clF1 0.09 / comp 0.38
R05N21 GT
baseline clF1 0.14 / comp 0.15
rgbd_s0 clF1 0.19 / comp 0.35
v2r_rgbd_s0 clF1 0.38 / comp 0.39
rgbd_64 clF1 0.43 / comp 0.53
R10N12 GT
baseline clF1 0.12 / comp 0.15
rgbd_s0 clF1 0.22 / comp 0.28
v2r_rgbd_s0 clF1 0.26 / comp 0.29
rgbd_64 clF1 0.33 / comp 0.36
R15N08 GT
baseline clF1 0.10 / comp 0.15
rgbd_s0 clF1 0.17 / comp 0.43
v2r_rgbd_s0 clF1 0.22 / comp 0.37
rgbd_64 clF1 0.12 / comp 0.46
R15N28 GT
baseline clF1 0.12 / comp 0.15
rgbd_s0 clF1 0.05 / comp 0.39
v2r_rgbd_s0 clF1 0.09 / comp 0.32
rgbd_64 clF1 0.09 / comp 0.33